Setting Up an AI Answering Bot with Google Sheets & Apps Script

Table of Contents

1. Introduction
2. Setting Up Your Form with JotForm
3. Connecting JotForm to Google Sheets
4. Setting Up Google Apps Script
5. Understanding the Code
6. Obtaining API Keys
7. Customizing the AI Prompt
8. Email Sending Options
9. Automating the Process
10. Troubleshooting
11. Conclusion

Introduction

This guide will walk you through setting up an automated AI answering bot that responds to form submissions with personalized AI-generated emails. The system works by:

1. Collecting information through a web form (JotForm)
2. Storing the data in Google Sheets
3. Processing the data with Google Apps Script
4. Generating personalized responses using AI (Claude)
5. Automatically sending emails to respondents

This solution is perfect for businesses or freelancers who want to automate initial responses to inquiries while maintaining a personalized touch.

Setting Up Your Form with JotForm

1. **Create a JotForm account**
    - Go to JotForm.com and sign up for an account
    - Note: You can use other form providers (TypeForm, Google Forms, etc.), but this guide focuses on JotForm
2. **Create a new form**
    - From your JotForm dashboard, click "Create Form"
    - Choose "Blank Form" or a template that suits your needs

3. **Add necessary form fields**

    - The following fields are recommended for our AI system:
        - First Name (Short Text)
        - Last Name (Short Text)
        - Email (Email field)
        - Company (Short Text)
        - Project Description (Long Text)
        - Timeline (Multiple Choice or Short Text)
        - Budget Range (Multiple Choice or Short Text)
        - Current Assets (Long Text or File Upload)
        - Additional Information (Long Text)
4. **Design and publish your form**

    - Customize the look and feel of your form
    - Add your branding elements
    - Click "Publish" to make your form live
    - Copy the form URL to share with potential clients

Connecting JotForm to Google Sheets

1. **Set up Google Sheets integration**

    - In your JotForm dashboard, open the form you created
    - Click on "Settings" > "Integrations"
    - Search for "Google Sheets" and select it
    - Click "Authenticate" and log in to your Google account
    - Grant necessary permissions
2. **Configure the integration**

    - Select "Create New Spreadsheet" or choose an existing one
    - Name your spreadsheet (e.g., "Client Inquiries")
    - Map your form fields to spreadsheet columns
    - Ensure all important fields are properly mapped
    - Click "Complete Integration"
3. **Verify the connection**

    - Submit a test entry through your form
    - Check your Google Sheets to confirm the data appears correctly
    - Make note of the exact spreadsheet name and column headers for the next steps

Setting Up Google Apps Script

1. **Open your Google Sheet**

    - Navigate to the Google Sheet connected to your JotForm
2. **Access Google Apps Script**

    - Click on "Extensions" in the top menu
    - Select "Apps Script"
    - A new tab will open with the Apps Script editor
3. **Create a new script file**

    - Delete any existing code in the editor
    - Copy and paste the code provided below
    - Rename the project (e.g., "AI Answering Bot")
4. **Configure the spreadsheet ID and sheet name**

    - In the code, locate the `SPREADSHEET_ID` variable
    - Replace the sample ID with your own spreadsheet ID (found in your spreadsheet URL)
    - Update the `SHEET_NAME` variable with the exact name of your sheet

Understanding the Code

The provided code consists of several functions that work together to:

1. **Parse submission data**: Extract and process form submissions
2. **Generate AI responses**: Use Claude AI to create personalized email content
3. **Send emails**: Deliver the AI-generated responses to clients

Let's examine the key components:

Core Functions

```
// Main function that processes spreadsheet rows and generates/sends responses
function sendAIRepliesWithResend() {
  // Spreadsheet setup, configuration, and column identification
  // For each row: parse data, generate AI response, and send email
}

// Parses budget information from various formats
function parseBudget(budgetString) {
  // Extracts numeric budget values from strings like "$1,000-$3,000"
}

// Converts column labels to camelCase for API use
function toCamel(label) {
  // Transforms "Project Overview" to "projectOverview"
}

// Calls the AI API to generate responses
function fetchAIResponse(systemPrompt, prompt) {
```

```
  // Sends request to Claude AI and processes the response
}

// Sends the email using Resend.com or alternative
function sendViaResend(to, subject, htmlBody, plainBody) {
  // Configures and sends the email
}
```

Important Variables to Configure

You'll need to update these variables in the code:

```
const SPREADSHEET_ID = "YOUR_SPREADSHEET_ID"; // Found in your spreadsheet URL
const SHEET_NAME = "YOUR_SHEET_NAME"; // The name of the tab in your spreadsheet
const MY_EMAIL = "your.email@example.com"; // Your email for "from" and "reply-to"
const BUDGET_THRESHOLD = 5000; // Budget threshold for requiring confirmation
const REPLICATE_API_KEY = "YOUR_REPLICATE_API_KEY"; // From replicate.com
const RESEND_API_KEY = "YOUR_RESEND_API_KEY"; // From resend.com (optional)
```

Obtaining API Keys

Replicate API Key for Claude

1. **Create a Replicate account**
    - Go to Replicate.com and sign up
    - Navigate to your account settings or dashboard
2. **Generate an API key**
    - Look for "API Tokens" or "API Keys" section
    - Click "Create API Token" or similar
    - Copy the generated API key
    - Replace `YOUR_REPLICATE_API_KEY` in the code with this value
3. **Claude model selection**
    - The code uses Claude 3.5 Sonnet by default
    - You can also use other Claude models like:
        - claude-3-opus (more capable but more expensive)
        - claude-3-haiku (faster and cheaper)
    - To change models, modify the URL in the `fetchAIResponse` function

Email Sending Options

Option 1: Resend.com (Recommended for professional emails)

1. **Create a Resend account**
    - Go to Resend.com and sign up
    - Verify your domain for better deliverability
2. **Generate an API key**
    - Navigate to API Keys section in your dashboard
    - Create a new API key
    - Copy the key and replace `YOUR_RESEND_API_KEY` in the code

Option 2: Gmail (Free alternative)

If you prefer to use Gmail instead of Resend, replace the `sendViaResend` function with:

```
function sendViaGmail(to, subject, htmlBody, plainBody) {
  try {
    GmailApp.sendEmail(
      to,
      subject,
      plainBody,
      {
        htmlBody: htmlBody,
        name: "Your Brand Name",
        replyTo: MY_EMAIL
      }
    );
    return true;
  } catch (error) {
    Logger.log(`Error sending email via Gmail: ${error}`);
    return false;
  }
}
```

And update the call in the main function:

```
const sendSuccess = sendViaGmail(emailTo, emailSubject, emailHtmlBody, emailPlainText);
```

Customizing the AI Prompt

The heart of your AI answering bot is the prompt that generates responses. You'll want to customize this to match your brand voice and business needs.

Here's a simplified example prompt you can customize:

```
userPrompt = `Generate a professional email reply to this potential client:
```

Name: ${name || 'Inquirer'}
Email: ${email}
Company: ${company || 'N/A'}
Project Description: ${desc || 'N/A'}
Timeline: ${timeline || 'N/A'}
Budget: ${budgetString || 'N/A'}
Assets: ${assets || 'N/A'}
Additional Info: ${extra || 'N/A'}

You are a [YOUR PROFESSION] working under the name "[YOUR BRAND]". Clients discover you through [YOUR MARKETING CHANNELS], where you showcase [YOUR SKILLS/SERVICES].

Your work spans [LIST YOUR SERVICES]. You're known for [YOUR UNIQUE SELLING PROPOSITION].

Goal: Write friendly, confident emails that show your expertise without sounding corporate. Be direct but warm. Keep responses under 180 words.

Tone & Voice:
- Write like you're messaging a smart, respectful colleague
- Keep it short and human
- Be warm and honest, not formal or hypey
- [ADD YOUR SPECIFIC TONE GUIDELINES]

Budget Approach:
- For limited budgets (< $${BUDGET_THRESHOLD}), focus on what's possible within constraints.
- For larger budgets (>= $${BUDGET_THRESHOLD}), be slightly more expansive in thinking.
- [ADD YOUR PRICING APPROACH]

Response Structure:
- Ask 2–3 pointed follow-up questions relevant to the budget/scope.
- [ADD YOUR PREFERRED EMAIL STRUCTURE]
- Keep the whole email body under 180 words.
- Always close with [YOUR SIGNATURE/BRAND].

Respond *only* in this structured JSON format (no other text):

```
{
  "subject": "...",
  "greeting": "...",
  "intro": "...",
  "projectOverview": "...",
  "questions": "...",
  "ideas": "...",
  "nextSteps": "...",
```

```
  "closing": "...",
  "signature": "Thanks,\\n[YOUR NAME]\\n[YOUR BRAND]"
}`;
```

Automating the Process

To have your script run automatically without manual intervention:

1. **Set up a time-based trigger**
    - In the Apps Script editor, click on "Triggers" (clock icon in the left sidebar)
    - Click "+ Add Trigger" at the bottom right
    - Configure the trigger:
        - Choose function: `sendAIRepliesWithResend`
        - Event source: "Time-driven"
        - Type of time: "Hour timer"
        - Hour interval: Select how often you want it to run (e.g., "Every hour")
    - Click "Save"
2. **Grant necessary permissions**
    - When setting up the trigger, you'll be prompted to authorize the script
    - Review and grant the requested permissions
3. **Test the automation**
    - Submit a test form entry
    - Wait for the trigger to run (or run the function manually once)
    - Check your email and the spreadsheet for results
4. **Remove the processing limit (optional)**
    - By default, the code processes one row per execution to avoid hitting quotas
    - To process all eligible rows, find and remove the `break;` line in the main loop
    - Be cautious as this may exceed Google's execution time limits for complex operations

Troubleshooting

Form data not appearing in spreadsheet:

- Verify your JotForm integration is properly configured
- Check mapping between form fields and spreadsheet columns
- Try submitting a test entry and check JotForm's submission logs

Script errors:

- Check the Execution logs in Apps Script (View > Execution Log)
- Verify API keys are correctly entered and not expired

- Ensure spreadsheet ID and sheet name match exactly

AI responses not generating:

- Check your Replicate API key and quota
- Review the prompt for any formatting issues
- Test the AI function separately to debug

Emails not sending:

- If using Resend, verify your API key and domain verification
- If using Gmail, check your daily sending quota (Keep it below 10 per day)
- Verify email addresses are correctly formatted

Conclusion

You now have a fully functional AI answering bot that automatically responds to client inquiries with personalized responses. This system:

- Saves you time by automating initial client communications
- Maintains a personalized touch through AI-generated responses
- Provides consistent messaging aligned with your brand
- Scales with your business without requiring additional staff

To further enhance your system, consider:

- Refining your AI prompt for even better responses
- Adding follow-up sequences for leads that don't respond
- Integrating with your CRM system
- Creating different response templates for different types of inquiries

Remember to regularly review the AI-generated responses to ensure they maintain the quality and tone you desire. The system will improve over time as you refine your prompts and processes.

This is the example code:

```
/**
 * Parses a budget string based on specific formats like "Label
($XXX-$YYY)" or "Label ($XXX+)".
 * Extracts the upper limit or single value for comparison.
 * @param {string} budgetString The budget string from the sheet
(e.g., "Standard ($1,000-$3,000)").
 * @return {number|null} The parsed numeric value (upper limit or
single value) or null.
 */
```

```
function parseBudget(budgetString) {
  if (!budgetString || typeof budgetString !== 'string') {
    return null;
  }
  try {
    let relevantPart = budgetString.replace(/^Budget range\?/i,
'').trim();
    const lastParenStart = relevantPart.lastIndexOf('(');
    const lastParenEnd = relevantPart.lastIndexOf(')');
    if (lastParenStart === -1 || lastParenEnd === -1 || lastParenEnd
< lastParenStart) {
       Logger.log(`No valid parenthesis group found at the end of:
"${relevantPart}"`);
       return null;
    }
    let content = relevantPart.substring(lastParenStart + 1,
lastParenEnd);
    content = content.replace(/[$\€£]/g, '');
    content = content.replace(/[.,]/g, '');
    const numbers = content.match(/\d+/g);
    if (numbers && numbers.length > 0) {
      const lastNumberStr = numbers[numbers.length - 1];
      return parseInt(lastNumberStr, 10);
    } else {
      Logger.log(`No numbers found in parenthesis content:
"${content}" from original "${budgetString}"`);
      return null;
    }
  } catch (e) {
    Logger.log(`Error parsing budget string "${budgetString}":
${e}`);
    return null;
  }
}

/**
 * Main function to process spreadsheet rows, generate AI
responses/summaries,
 * and send emails based on budget and confirmation.
 */
function sendAIRepliesWithResend() {
```

```
  // --- Spreadsheet Setup ---
  const SPREADSHEET_ID = "YOUR_SPREADSHEET_ID"; // !!! IMPORTANT:
Enter your Spreadsheet ID here !!!
  const SHEET_NAME = "YOUR_SHEET_NAME"; // The name of your sheet
tab (e.g., "Form Responses")
  const sheet =
SpreadsheetApp.openById(SPREADSHEET_ID).getSheetByName(SHEET_NAME);
  if (!sheet) {
      Logger.log(`Error: Sheet "${SHEET_NAME}" not found in
Spreadsheet ID "${SPREADSHEET_ID}".`);
      return;
  }
  const data = sheet.getDataRange().getValues();
  let headers = data[0];

  // --- Configuration ---
  const MY_EMAIL = "your.email@example.com"; // Your email (used for
'from' and 'reply_to')
  const BUDGET_THRESHOLD = 5000; // Threshold for requiring
confirmation

  // --- Get Column Indices ---
  const columnIndices = {};
  const requiredColumns = ["First Name", "Last Name", "Email",
"Company", "Project Description",
                           "What's your ideal timeline?", "Budget
range?", "Current brand assets?",
                           "Anything else you'd like to share about
your project?"];
  requiredColumns.forEach(colName => {
      const index = headers.indexOf(colName);
      if (index === -1) {
          Logger.log(`Error: Required column "${colName}" not found
in sheet headers.`);
          // Consider throwing an error or using
SpreadsheetApp.getUi().alert()
          // throw new Error(`Required column "${colName}" not
found.`);
          return; // Stop if critical column missing
      }
      columnIndices[colName] = index;
```

```
  });
   // Rename for easier access
  const firstNameIndex = columnIndices["First Name"];
  const lastNameIndex = columnIndices["Last Name"];
  const emailIndex = columnIndices["Email"];
  const companyIndex = columnIndices["Company"];
  const descIndex = columnIndices["Project Description"];
  const timelineIndex = columnIndices["What's your ideal
timeline?"];
  const budgetIndex = columnIndices["Budget range?"];
  const assetsIndex = columnIndices["Current brand assets?"];
  const infoIndex = columnIndices["Anything else you'd like to share
about your project?"];

  // --- Ensure Status, Confirmation, and Reply Columns Exist ---
  let statusIndex = headers.indexOf("Status");
  if (statusIndex === -1) {
    Logger.log("Status column not found, adding it.");
    sheet.getRange(1, headers.length + 1).setValue("Status");
    statusIndex = headers.length;
    headers.push("Status");
  }

  let confirmIndex = headers.indexOf("Send Confirmation?");
  if (confirmIndex === -1) {
    Logger.log("Column 'Send Confirmation?' not found, adding it.");
    sheet.getRange(1, headers.length + 1).setValue("Send
Confirmation?");
    confirmIndex = headers.length;
    headers.push("Send Confirmation?");
  }

  let replyReceivedIndex = headers.indexOf("Reply Received?");
  if (replyReceivedIndex === -1) {
    Logger.log("Column 'Reply Received?' not found, adding it.");
    sheet.getRange(1, headers.length + 1).setValue("Reply
Received?");
    replyReceivedIndex = headers.length;
    headers.push("Reply Received?");
  }
```

```
  // --- Ensure AI Response Field Columns Exist ---
  const fieldNames = ["Subject", "Greeting", "Intro", "Project
Overview", "Questions", "Ideas", "Next Steps", "Closing",
"Signature"];
  const fieldIndexes = {};
  fieldNames.forEach((field) => {
    let index = headers.indexOf(field);
    if (index === -1) {
      Logger.log(`Column "${field}" not found, adding it.`);
      sheet.getRange(1, headers.length + 1).setValue(field);
      index = headers.length;
      headers.push(field);
    }
    fieldIndexes[field] = index;
  });

  // --- Process Rows ---
  for (let rowIndex = 1; rowIndex < data.length; rowIndex++) {
    const row = data[rowIndex];
    // Ensure row has enough columns (use the highest index needed)
    const maxIndex = Math.max(emailIndex, statusIndex, budgetIndex,
confirmIndex, replyReceivedIndex, ...Object.values(fieldIndexes));
    if (row.length <= maxIndex) {
        // Logger.log(`Skipping row ${rowIndex + 1}: Not enough
data.`);
        continue;
    }

    const email = row[emailIndex];
    const status = row[statusIndex] ?
String(row[statusIndex]).toLowerCase().trim() : "";

    // Skip if no email OR if already sent OR if it's high budget
and already sent
    const processedStatuses = ["sent", "sent_high_budget",
"send_failed", "reply_failed", "reply_content_error",
"follow_up_sent"]; // Add statuses that mean "don't process again"
    if (!email || processedStatuses.includes(status)) {
        // Logger.log(`Skipping row ${rowIndex + 1}: No email or
already processed/failed (Status: ${status})`);
        continue;
```

```
    }

    // Allow processing if status is 'awaiting_confirmation' or
empty/new
    if (status === "awaiting_confirmation") {
      Logger.log(`Row ${rowIndex + 1}: Re-checking status
'awaiting_confirmation'.`);
    }

    // --- Get Client Data ---
    const name = `${row[firstNameIndex] || ''} ${row[lastNameIndex]
|| ''}`.trim();
    const company = row[companyIndex] || '';
    const desc = row[descIndex] || '';
    const timeline = row[timelineIndex] || '';
    const budgetString = row[budgetIndex] || '';
    const assets = row[assetsIndex] || '';
    const extra = row[infoIndex] || '';

    // --- Parse Budget ---
    const budgetValue = parseBudget(budgetString);
    const isHighBudget = budgetValue !== null && budgetValue >=
BUDGET_THRESHOLD;
    Logger.log(`Row ${rowIndex + 1}: Email=${email}, Budget
String='${budgetString}', Parsed Value=${budgetValue}, Is High
Budget=${isHighBudget}`);

    // --- Define AI Prompts (Client Email Prompt is now always
used) ---
    const systemPrompt = "You are a friendly, creative but
professional freelance professional who responds to client inquiries
by email.";
    let userPrompt;
    let aiResult;

    // --- Generate Client Email Content (unless already generated
and waiting) ---
    // Check if AI fields are already populated (e.g., from a
previous run that was awaiting confirmation)
```

```
    const needsAIGeneration = !row[fieldIndexes["Subject"]] &&
status !== "awaiting_confirmation"; // Only generate if Subject is
empty AND not already waiting

    if (needsAIGeneration) {
        Logger.log(`Row ${rowIndex + 1}: Generating AI response for
client.`);
        userPrompt = `Generate a professional email reply to this
potential client:

Name: ${name || 'Inquirer'}
Email: ${email}
Company: ${company || 'N/A'}
Project Description: ${desc || 'N/A'}
Timeline: ${timeline || 'N/A'}
Budget: ${budgetString || 'N/A'} (${budgetValue !== null ? 'Parsed
as approx: ' + budgetValue : 'Could not parse'})
Assets: ${assets || 'N/A'}
Additional Info: ${extra || 'N/A'}

You are a creative professional working under the name "[YOUR BRAND
NAME]". Clients discover you through your marketing channels, where
you showcase your skills and expertise. Your work spans [LIST YOUR
SERVICES/SKILLS]. You're known for translating complex ideas into
clear, emotional, and conversion-oriented results.

You've worked with brands in [YOUR INDUSTRIES]—often helping them
improve their business outcomes through your services.

Goal: Write friendly, confident emails that show your expertise
without sounding corporate. Be direct but warm. Don't overpromise or
try too hard to impress. Keep responses grounded in what's realistic
and under 180 words.

Tone & Voice:
- Write like you're messaging a smart, respectful colleague
- Keep it short and human
- Keep the language easy
- Be warm and honest, not formal or hypey
- Avoid clichés or filler phrases
```

```
- Never repeat the client's info or restate their idea unless it
adds value

Budget Approach:
- For limited budgets (< $${BUDGET_THRESHOLD}), focus on what's
possible within constraints.
- For larger budgets (>= $${BUDGET_THRESHOLD}), you can be slightly
more expansive in thinking but still keep the initial reply concise
and focused on next steps (like a call).
- Never offer free work or trial services.

Response Structure:
- Ask 2–3 pointed follow-up questions relevant to the budget/scope.
- If fitting, add one sharp direction suggestion (not a full
concept).
- Keep the intro and overview very short (5–10 words).
- Keep the whole email body under 180 words.
- Always close with reference to your brand.

Respond *only* in this structured JSON format (no other text):

{
  "subject": "...",
  "greeting": "...",
  "intro": "...",
  "projectOverview": "...",
  "questions": "...",
  "ideas": "...",
  "nextSteps": "...",
  "closing": "...",
  "signature": "Thanks,\\n[YOUR NAME]\\n[YOUR BRAND]"
}`;

      aiResult = fetchAIResponse(systemPrompt, userPrompt);

      if (!aiResult) {
        Logger.log(`⚠️ AI failed to generate reply for lead:
${email}`);
        sheet.getRange(rowIndex + 1, statusIndex +
1).setValue("reply_failed");
        continue; // Skip this row
```

```
      }

      // Validate expected fields & update sheet
      try {
          fieldNames.forEach((field) => {
              const key = toCamel(field);
              const colIndex = fieldIndexes[field];
              const isOptional = ["ideas"].includes(key); // 'ideas'
can be empty
              let value;

              if (aiResult.hasOwnProperty(key)) {
                  value = aiResult[key] || ""; // Use empty string
if key exists but value is null/undefined/empty
              } else if (!isOptional) {
                  Logger.log(`⚠️ AI response missing required key:
${key} for ${email}. Using placeholder.`);
                  value = `[Missing Required: ${key}]`; //
Placeholder
              } else {
                  value = ""; // Optional field missing, treat as
empty
              }
              aiResult[key] = value; // Ensure the key exists in
aiResult for later use, even if empty or placeholder

              // Update sheet cell
              if (colIndex !== undefined) {
                  sheet.getRange(rowIndex + 1, colIndex +
1).setValue(value);
              }
          });
          Logger.log(`Row ${rowIndex + 1}: AI response generated and
populated in sheet.`);
      } catch(e) {
           Logger.log(`Error updating sheet with AI response for row
${rowIndex + 1}: ${e}`);
           sheet.getRange(rowIndex + 1, statusIndex +
1).setValue("reply_content_error");
           continue; // Don't try to send if sheet update failed
      }
```

```
    } else {
        // AI fields were already populated, likely waiting for
confirmation.
        // Reconstruct the aiResult object from the sheet values for
sending.
        Logger.log(`Row ${rowIndex + 1}: AI response already exists
in sheet. Reconstructing for potential send.`);
        aiResult = {};
        fieldNames.forEach(field => {
            const key = toCamel(field);
            aiResult[key] = row[fieldIndexes[field]] || ""; // Read
value from sheet
        });
        // Basic check if essential parts are missing even from
sheet
        if (!aiResult.subject || !aiResult.greeting ||
!aiResult.signature) {
             Logger.log(`⚠️ Reconstructed AI response from sheet is
missing essential fields for row ${rowIndex + 1}. Status:
${status}`);
             sheet.getRange(rowIndex + 1, statusIndex +
1).setValue("reply_content_error");
             continue;
        }
    }

    // --- Prepare Email Content ---
    const emailTo = email; // Always send to the client email now
    const emailSubject = aiResult.subject || `Following up on your
inquiry`; // Fallback subject
    const emailPlainText = [
        aiResult.greeting, "",
        aiResult.intro, "",
        aiResult.projectOverview, "",
        (aiResult.questions &&
!aiResult.questions.startsWith("[Missing")) ? "Questions:\n" +
aiResult.questions : null, "",
        (aiResult.ideas && !aiResult.ideas.startsWith("[Missing")) ?
aiResult.ideas : null, "", // Removed "Ideas:" label if present
        aiResult.nextSteps, "",
        aiResult.closing, "",
```

```
        aiResult.signature
    ].filter(line => line !== null).join("\n").replace(/\n{3,}/g,
'\n\n');
    const emailHtmlBody = emailPlainText.replace(/\n{2,}/g,
"<br><br>").replace(/\n/g, "<br>");

    // --- Conditional Sending Logic ---
    let shouldSend = false;
    let finalStatus = "";
    let finalColor = "";

    if (isHighBudget) {
        const confirmationStatus = row[confirmIndex] ?
String(row[confirmIndex]).toLowerCase().trim() : "";
        Logger.log(`Row ${rowIndex + 1}: High budget lead. Checking
confirmation status: '${confirmationStatus}'`);
        if (confirmationStatus === "yes" || confirmationStatus ===
"ja") {
            shouldSend = true;
            finalStatus = "sent_high_budget";
            finalColor = "#ccffcc"; // Light Green for sent high
budget
            Logger.log(`Row ${rowIndex + 1}: Confirmation received.
Proceeding to send.`);
        } else {
            shouldSend = false;
            finalStatus = "awaiting_confirmation";
            finalColor = "#fffacd"; // Light yellow for waiting
            Logger.log(`Row ${rowIndex + 1}: Awaiting confirmation.
Email will not be sent yet.`);
            // Update status immediately if waiting
            try {
                sheet.getRange(rowIndex + 1, statusIndex +
1).setValue(finalStatus);
                sheet.getRange(rowIndex + 1, 1, 1,
sheet.getLastColumn()).setBackground(finalColor);
            } catch (e) {
                Logger.log(`Error updating sheet status/color while
awaiting confirmation for row ${rowIndex + 1}: ${e}`);
            }
        }
```

```
    } else {
        // Low budget, send immediately
        shouldSend = true;
        finalStatus = "sent";
        finalColor = "#e0f7df"; // Light green/blue for standard
sent
        Logger.log(`Row ${rowIndex + 1}: Low budget lead. Proceeding
to send immediately.`);
    }

    // --- Send Email (choose one method: Resend or Gmail) ---
    if (shouldSend) {
      if (emailTo && emailSubject && (emailHtmlBody ||
emailPlainText)) {
        Logger.log(`Attempting to send email to ${emailTo} with
subject "${emailSubject}"`);

        // OPTION 1: Send via Resend.com
        const sendSuccess = sendViaResend(emailTo, emailSubject,
emailHtmlBody, emailPlainText);

        // OPTION 2: Send via Gmail (uncomment to use this instead)
        // const sendSuccess = sendViaGmail(emailTo, emailSubject,
emailHtmlBody, emailPlainText);

        if (sendSuccess) {
          // Status and color already determined above
          try {
              sheet.getRange(rowIndex + 1, statusIndex +
1).setValue(finalStatus);
              sheet.getRange(rowIndex + 1, 1, 1,
sheet.getLastColumn()).setBackground(finalColor);
              Logger.log(`✅ Email successfully sent to ${emailTo}
(Status: ${finalStatus})`);
          } catch (e) {
               Logger.log(`Error updating sheet status/color for row
${rowIndex + 1} after successful send: ${e}`);
          }
        } else {
          Logger.log(`❌ Sending failed for ${emailTo}.`);
          const failedStatus = "send_failed";
```

```
            try {
               sheet.getRange(rowIndex + 1, statusIndex +
1).setValue(failedStatus).setBackground("#f4cccc"); // Light red
            } catch (e) {
                Logger.log(`Error updating sheet status for row
${rowIndex + 1} after failed send: ${e}`);
            }
        }
      } else {
        Logger.log(`❌ Email content missing or invalid for row
${rowIndex + 1} (${email}). Cannot send.`
```